

1. Read V_1 sinim V_2 & V_1 sinim hd mouse brain data.

2688 x 18078 spots genes
6296088 x 19059 bins genes.

2. Identify common genes & subset the dataset

a. Preprocess dataset.

- (i) ensure gene names are unique.
- (ii) Filter out genes that are minimally expressed in different cells.

Gene Prefiltering

Annotation A cell-by-gene matrix $X \in \mathbb{R}^{n \times G}$ retain only genes expressed in at least m cells

sc.pp.filter_genes (adata, min.cells = 3).

- (ii) Highly variable genes selection and normalization per cell

Sc. pp. highly - variable - genes (---, n_top_genes = 1000).

sc.pp.normalize_per_cell(adata) each cell $\tilde{X}(i, g) = \frac{X(i, g)}{\sum_g X(i, g)} \times \text{scaler factor}$

sc. pp. longip. (adventu).

- b. Compute intersection genes. using `np.intersect1d(~var_names, ~var_names)`

Subset V_2 HD

$$2688 \times 14842 \quad 6132829 \times 14842$$

3. Generate spatial neighbour network. using Cal-Spatial-Net() from **SpaGRA**

Given AnnData with spatial coordinates $C = \{(x_i, y_i)\}_{i=1}^N$ `adata.obs['spatial']`

Define $G = (V, E)$ where each node $i \in V$ corresponds to a cell, and an undirected

$$E = \{(i, j) \mid j \in N(i)\}$$

edge $(i, j) \in E$ exists if the Euclidean distance $d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} < r$.

⇒ build dataframe of edges with column ['cell1', 'cell2', 'Distance'] in
 (i, j) d(i, j).

```
adata.uns['Spatial-Net'].
```

Since hol data is large, randomly sample a subset with 10000 cells
np.random.choice.

code: Nearest Neighbors (with radius = rad_cutoff).

4. Graph & Feature Matrix. Transfer_data(addata) from SpaGRA

ca) Using the Constructed spatial network, convert it into graph representation and extract feature matrix.

Define the sparse adjacency matrix $A \in \mathbb{R}^{n \times n}$

$$\Rightarrow A_{ij} = \begin{cases} 1 & \text{if } (i,j) \in E \text{ (or } i=j, \text{ for self-loop } \Rightarrow A_{ii} \leq 1). \\ 0 & \text{otherwise.} \end{cases}$$

For each edge populate the sparse matrix $A = \sum_{(i,j) \in E} e_i e_j^T$, where e_i is the standard basis vector with 1 in the i th position.

cb) Feature matrix X .

Anndata contains gene expression matrix $X \in \mathbb{R}^{n \times \overset{\text{\# Genes}}{d}}$, $\Rightarrow X \in \mathbb{R}^{n \times \overset{\text{common set}}{d}}$

each cell $i \Rightarrow X_i = [X_{i1}, \dots, X_{id}] \in \mathbb{R}^d$.

$X = \text{np.array(addata.X)}$ then alignment with Graph A .

Row i of X same with row/column i in the adjacency matrix A .

Dual Graph Convolutional Autoencoder (GCAE)

1. Input: adjacency $A \in \mathbb{R}^{N \times N}$ $A^{(v_2)} \in \mathbb{R}^{N_{v_2} \times N_{v_2}}$ $A^{(hd)} \in \mathbb{R}^{N_{hd} \times N_{hd}}$
feature $X \in \mathbb{R}^{N \times d}$ $X^{(v_2)} \in \mathbb{R}^{N_{v_2} \times d}$ $X^{(hd)} \in \mathbb{R}^{N_{hd} \times d}$

2 GC Encoder

- ① Single GC $GC(X, A; W) = AXW$ $W \in \mathbb{R}^{d_{in} \times d_{out}}$ learned weight matrix.

↓ support = XW output = AXW
torch.matmul(X , self.weight) torch.matmul(A , support).

- ② Two-layer encoder to obtain a latent representation Z in lower-dimensional space.

First layer: $H = \text{LeakyReLU}(AXW_0)$ $h = \text{self.leaky_relu}(\text{self.gc}(X, A))$

Second layer: $Z = \text{leakyReLU}(AHW_1)$ $z = \text{self.leaky_relu}(\text{self.gc}(h, A))$

Two datasets $\Rightarrow$ two separate encoders

Visum v2: $Z^{(v_2)} = \phi(A^{(v_2)} \phi(A^{(v_2)} X^{(v_2)} W_0^{(v_2)})) W_1^{(v_2)}$

Visum hd: $Z^{(hd)} = \phi(A^{(hd)} \phi(A^{(hd)} X^{(hd)} W_0^{(hd)})) W_1^{(hd)}$ shared encoder So using same W_0, W_1

- ③ Decoder reconstructs the adjacency matrix from latent matrix embeddings Z .

$\hat{A} = \sigma(ZZ^T)$ sigmoid squashes the value to $[0, 1]$. decode (Z).

$\Rightarrow \hat{A}_{v_2} = \sigma(Z_{v_2} Z_{v_2}^T), \hat{A}_{hd} = \sigma(Z_{hd} Z_{hd}^T)$

↓
force $Z^{(v_2)}$
and $Z^{(hd)}$ into
same L -dimensional
space.

④ Loss Function

- i) Using binary cross-entropy between predict $\hat{A}$ and real adjacency A .

loss = F.binary_cross_entropy(A_{hat}, A).

$\mathcal{L} = -\sum_{i,j} [A_{ij} \log(\hat{A}_{ij}) + (1 - A_{ij}) \log(1 - \hat{A}_{ij})]$

$\mathcal{L}_{\text{recon}} = \text{BCE}(A_{v_2}, \hat{A}_{v_2}) + \text{BCE}(A_{hd}, \hat{A}_{hd})$

- ii) Alignment loss to promote similarity between Z_{v_2} and Z_{hd} .

$\mathcal{L}_{\text{align}} = \|\text{mean}(Z_{v_2}) - \text{mean}(Z_{hd})\|_2^2$

So total loss = $\mathcal{L}_{\text{recon}} + \alpha \mathcal{L}_{\text{align}}$ hyperparameter balance two losses
remove when joint latent loss

Training model.

Dual GCAE (model)

training loop.

optimizer with $\eta = 1e^{-3}$ $\Rightarrow$

$$\lambda^{\text{align}} = 0.1$$

$$\text{epochs} = 500$$

Shared : For each dataset $d \in \{V_2, \text{hd}\}$

$$z^{(d)} = f(X^{(d)}, A^{(d)}; W_0, W_1)$$

$$\hat{A}^{(d)} = \sigma(z^{(d)} z^{(d)T})$$

$$\mathcal{L} = \sum_d \text{BCE}(A^{(d)}, \hat{A}^{(d)}).$$

$\Downarrow$

$$z_d \in \mathbb{R}^{(N_d \times d)}.$$

008 bins resolution.

batch effect on encoders

scv1.